PROJECT SYNOPSIS · 2026

CampusBazaar

Your Campus. Your Marketplace.

A student-focused university marketplace and campus community platform for exchanging useful items, discovering campus deals, connecting with sellers, and sharing local notices.

Project title

CampusBazaar: A Student-Only University Marketplace and Campus Community Platform

Application type

Full-stack web application

Core stack

React · Spring Boot · Java 17 · Spring Security · JPA · MySQL

Executive Summary

CampusBazaar brings student-to-student commerce and campus information into one university-oriented web application.

Abstract

Students regularly need affordable textbooks, electronics, furniture, and other campus essentials. At the same time, other students have useful items they can sell, rent, exchange, or give away. CampusBazaar provides a focused place for discovering these items within a student community, communicating with sellers, and managing listings.

The application combines a responsive React frontend with a Java Spring Boot REST API. Spring Data JPA persists marketplace and community records in a relational database. Spring Security and JWT bearer tokens protect account-specific operations, while service-level ownership checks prevent users from changing other students' listings or posts. The backend is configured for MySQL; automated integration tests run against isolated in-memory H2.

Beyond commerce, the platform includes verified-student indicators, listing reports, administrator moderation, buyer–seller messaging, notifications, a browser-persistent wishlist, and a university-scoped board for lost items, found items, and announcements.

Problem Addressed

General marketplaces are broad and may not make it easy to find items near a particular campus or establish community trust. Campus notices can also become fragmented across chats, physical boards, and social platforms. Students consequently spend time searching, arranging exchanges, and verifying whether a listing or notice is relevant to their university.

> **Project premise:** make student commerce more discoverable and campus communication more organized, while enforcing account, role, and ownership rules in the backend.

Project Outcome

The current implementation provides an end-to-end foundation: students can browse real listing records, inspect details, authenticate, publish and manage their own listings, communicate with listing participants, report suspicious content, and share campus notices. Administrators have a protected moderation surface.

Objectives & Project Scope

Objectives

1. Deliver a responsive, accessible student marketplace experience.
2. Make listings discoverable by query, category, listing type, price, sorting, and pagination.
3. Allow authenticated users to publish, update, and remove their own available listings.
4. Support sale, rental, exchange, and giveaway transaction types.
5. Enable secure buyer–seller communication with persisted messages and notifications.
6. Provide a wishlist that remains available after refresh in the same browser.
7. Give users a way to report listings and administrators a protected moderation dashboard.
8. Support university-specific Lost, Found, and campus announcement posts.
9. Protect private account data and ensure authorization is enforced server-side.

Primary User Groups

Student buyer

Browses listings, searches and filters items, views seller and item details, saves favourites, and contacts a seller.

Student seller

Creates listings, manages their own available listings, responds to prospective buyers, and marks campus notices resolved or removed.

Campus community member

Uses the authenticated university-scoped board to share Lost, Found, and announcement posts.

Administrator

Reviews reports, moderates listings, inspects masked student identity data, and performs account verification or status actions.

Scope Boundaries

The system focuses on listing discovery, student interactions, and campus notices. It is not a payment processor: buyers and sellers arrange the transaction themselves. Listing photographs currently support a local selection preview only; uploads are not stored. Wishlist persistence is browser-local rather than account-synchronized.

Data principle: preserve existing database records and apply listing removal as a soft status change, not a destructive database delete.

Technology & Architecture

Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	Responsive single-page user interface and production bundling.
Client state and routing	Redux Toolkit, React Router	Session and wishlist state, page navigation.
HTTP client	Axios	API requests, bearer-token attachment, and readable error messages.
Backend	Java 17, Spring Boot 3, Spring MVC	REST endpoints and application services.
Persistence	Spring Data JPA, Hibernate	Relational entity mapping and repository access.
Security	Spring Security, JWT	Stateless authentication, roles, and protected requests.
Database	MySQL configured; H2 for tests	Durable application records and isolated integration tests.
Verification	Vitest, React Testing Library, Spring Boot Test	UI tests and backend integration tests.

Request & Persistence Flow

1. The React application presents a marketplace page or authenticated workflow.
2. Axios sends an HTTP request to the Spring Boot API. During local development, Vite proxies `/api` to port 8080.
3. Spring Security checks whether the endpoint is public or requires a valid bearer token and authority.
4. Controllers validate request shapes and call services; services enforce business rules and resource ownership.
5. Spring Data repositories read or persist entity records through JPA/Hibernate.
6. The API returns a standard response envelope; the frontend renders results, validation errors, or retry states.

Environment Configuration

Database URL and credentials, JWT signing secret, CORS origins, seed-data setting, and administrator credentials are configured through backend environment variables. Secrets belong in the backend runtime environment and must not be embedded in frontend code or committed to source control.

Marketplace & Discovery

Browse and View

The marketplace homepage presents listing cards and campus-focused sections using listing data returned by the backend. Cards can show title, rupee price, condition, category, seller verification, deal labeling, and an image when an image URL exists. Selecting a listing opens a detail page with its description, transaction type, seller summary, and safety guidance.

Search, Filters, Sorting & Pages

Discovery is provided by the server-side `GET /api/listings/search` endpoint. Query terms can match listing title, description, category, or seller. Optional parameters narrow results by category slug, minimum and maximum price, or listing type.

Parameter	Role	Notes
`q`	Text search	Matches title, description, category, and seller name.
`category`	Category filter	Accepts a category slug and includes its child listings.
`minPrice`, `maxPrice`	Price range	Rejects a minimum greater than the maximum.
`listingType`	Transaction filter	Sale, rent, exchange, or giveaway.
`sort`	Sort order	`newest`, `price_asc`, or `price_desc`.
`page`, `size`	Pagination	Zero-based page; size is bounded to at most 50.

Marketplace Sections

The homepage supports recently listed items, popular items, budget finds, student deals, and hostel clearance using listing records and their backend attributes. Search and category controls provide focused result sets; the server, rather than a client-side mock collection, performs the paginated discovery query.

> The existing public list and detail reads remain available. Search adds a paginated endpoint rather than replacing existing API contracts.

Category Data

Categories are stored in the backend database and exposed through a public read endpoint. The sell form loads active categories from this API, and the backend validates the selected category again during create or edit.

Listing Lifecycle

Create a Listing

A signed-in student submits a title, description, active category, condition, transaction type, and applicable price. The API derives the seller from the authenticated JWT subject; the client cannot assign another seller. Sale and rent require positive prices. Exchange and giveaway listings are normalized to zero price on the server.

Edit and Remove

The listing detail page exposes edit and remove controls only when the current account matches the listing seller. The backend independently verifies ownership on every mutation, rejects changes to demo records and non-available records, and validates the replacement category and price.

Removal is a soft delete: the listing status changes to `REMOVED`, preserving the row and its relationships. Public available-listing reads then omit it. This approach retains database history and avoids unnecessary destructive operations.

End-to-End Lifecycle

1. Student opens the sell form and loads active category options from the backend.
2. Client validates required fields and provides immediate field-level browser feedback.
3. Backend validates request shape, account status, category, listing type, and price.
4. Backend persists a listing linked to the authenticated seller and selected category.
5. Owner opens details to edit, or requests soft removal; non-owners are rejected.
6. Marketplace reads and search reflect only available records.

Image Handling

The form currently accepts selected JPG, PNG, or WebP files for preview and checks the preview file size. This is not an upload workflow: no file is sent to the backend or stored durably, and a newly created listing does not acquire an uploaded image URL. Production image hosting and content validation remain future work.

> **Persistence status:** H2 runtime verification exercised authenticated listing creation, update, and soft deletion. Live MySQL verification remains outstanding because no local MySQL service was available during this work.

Accounts, Trust & Communication

Accounts and Authorization

Student registration associates an account with a university and starts the account in a pending verification state. Login returns a JWT used by protected API calls. Authentication is stateless. Admin authority is represented by the backend role; frontend route or navigation checks are presentation conveniences and do not replace API authorization.

The backend checks current account status when validating JWT-authenticated operations. Suspended accounts cannot continue making authenticated requests with previously issued tokens. Student identity data is minimized in responses: full student IDs and password hashes are not exposed to public or general user clients.

Trust & Safety Controls

- Verified-student status can be surfaced as a seller trust indicator.
- Authenticated users can report suspicious listings.
- Administrators receive reports through a protected moderation surface.
- Ownership checks prevent one student from editing or removing another student's listing.
- Campus-board reads are authenticated and scoped to the viewer's university.
- Users are encouraged to arrange safe, on-campus handovers and keep discussions on-platform.

Buyer–Seller Messaging

A buyer can start a conversation from an available listing. The conversation is tied to the listing and its buyer/seller participants. Only participants can retrieve messages or send new ones. Sending a message creates a notification for the other participant; opening a conversation updates unread notification state.

The inbox and message views poll periodically for updates. This is a working persistence-based approach, but it is not a real-time WebSocket delivery architecture.

Wishlist

Wishlist selection is persisted in browser local storage. It survives refreshes in that browser and allows quick saving from product cards. At present, it is not associated with the student's account and does not synchronize between browsers or devices.

> Security decisions are enforced at the backend: endpoint authentication, administrator roles, conversation participation, account state, and resource ownership.

Administration & Campus Board

Protected Administration

The administration dashboard and `/api/admin/**` endpoints require the administrator role. Administrators can inspect dashboard summaries, review masked student identity details, verify students, suspend or reinstate accounts, moderate listings, and process user-submitted reports.

Account and moderation actions are executed by the backend. Administrator secrets are configured in the backend environment. The sample local-development account is not suitable for production and must be replaced with a strong private credential.

Campus Board

The campus board extends the marketplace with university-scoped community notices. Post types include:

Lost item

A student can describe an item they are seeking and where it was last seen.

Found item

A student can describe an item found on campus and its approximate location.

Announcement

A student can share relevant information with their university community.

Resolution

The post owner can mark an open notice resolved or remove their post.

Campus posts are associated with the poster's account and university on the server, rather than accepting arbitrary author or university identifiers from the client. The board is not public: users must authenticate, and reads are filtered by the account's university. Response objects avoid email addresses, phone numbers, and student IDs.

Data and Privacy Considerations

- Do not include private contact information or student IDs in post descriptions.
- Use platform conversations when arranging a handover.
- Moderation reports remain private to administrators.
- Listing soft deletion preserves the record while removing it from the public available set.

Data Model & API Surface

Core Domain Entities

Entity	Responsibilities and relationships
User	Account identity, role, status, verification state, and university association.
University	University identity used for account association and campus-board scope.
Category	Active listing categories organized as parent categories and subcategories.
Listing	Item title, description, price, condition, type, status, category, and seller.
Conversation	Listing-specific buyer/seller participant relationship.
ChatMessage	Persisted content, sender, and conversation association.
UserNotification	Persisted user notification and read/unread state.
ListingReport	Reported listing, reporter, reason, and moderation status.
CampusPost	University-scoped notice, author, type, location, and resolution status.

Representative API Groups

API group	Example operations	Access model
Listings	`GET /api/listings`, `GET /api/listings/search`, `POST /api/listings`, `PUT/DELETE /api/listings/{id}`	Reads public; writes authenticated and owner-checked.
Categories / universities	`GET /api/categories`, `GET /api/universities`	Public read.
Authentication	`POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`	Register/login public; current account requires JWT.
Conversations	`/api/conversations`, `/messages`	Authenticated participants only.
Reports / moderation	`/api/reports`, `/api/admin/**`	Authenticated submission; administrator review.
Campus board	`/api/campus-board`	Authenticated; university scoped; owner-managed changes.

Persistence Approach

JPA entities and repositories map the domain to relational records. Schema evolution is configured additively through Hibernate's update mode. Demo data seeding is conditional and runs only when the corresponding table is empty. Automated tests use H2 in a test profile and do not reset or connect to the user's MySQL database.

Verification, Limitations & Next Steps

Verification Performed

- ✓ Backend `mvn clean package`: 13 integration tests passed against H2.
- ✓ Frontend `npm test -- --run`: 20 tests passed across 8 test files.
- ✓ Frontend `npm run build`: production build succeeded.
- ✓ Isolated H2 runtime: health, categories, paginated search, and campus-board reads returned HTTP 200.
- ✓ Anonymous listing creation was rejected with HTTP 401.
- ✓ Authenticated listing creation, owner update, soft deletion, and campus-post creation/resolution were exercised at runtime.

Pending: local MySQL service was unavailable during verification. Port 3306 refused connections, and no MySQL/MariaDB Windows service was registered. Therefore, live MySQL connectivity and writes to that database are not claimed as verified.

Current Limitations

- Listing images are locally previewed only and are not uploaded or durably stored.
- Wishlist data is stored per browser rather than synchronized with an account.
- Chat and notification updates use periodic polling rather than WebSockets.
- Campus board has no attachments, comment threads, or direct reply feature.
- Live MySQL runtime verification requires an available database and valid local environment configuration.

Recommended Future Work

1. Enable image uploads with server-side type/size validation and durable object storage.
2. Move wishlists to account-backed persistence and synchronize across devices.
3. Add WebSocket messaging and notification delivery where deployment infrastructure supports it.
4. Verify MySQL startup, schema evolution, CRUD operations, and preservation of existing records in the deployment environment.
5. Expand campus services, exchange proposals, rental requests, and deal or coupon workflows.
6. Continue improving accessibility, monitoring, rate limits, and production security configuration.

Conclusion

CampusBazaar is a full-stack foundation for student commerce and campus notices. Its current code supports real backend-backed listings, secure owner actions, paginated discovery, participant-scoped messaging, reporting and administration, and university-scoped posts. The automated suite and isolated runtime confirm these flows against H2; production MySQL validation and the listed feature extensions remain the next steps.